

PONDICHERRY UNIVERSITY



SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

Name : PILLI DHEERAJ

Register Number : 23384119

Semester : VI

Subject : Distributed Systems

Subject code : CSCS308

PONDICHERRY UNIVERSITY
(A CENTRAL UNIVERSITY)



SCHOOL OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE

B.Sc. Computer Science (Honors)

PRACTICAL LAB RECORD

BONAFIDE CERTIFICATE

This is to certify that this is a Bonafide record of practical work done by
PILLI DHEERAJ, Reg.No. 23384119 during semester VI of the year 2025-2026

EXAMINATION DATE

FACULTY IN-CHARGE

TABLE OF CONTENTS

PRACTICAL NO	INDEX
1	MESSAGE PASSING BETWEEN TWO PROCESSES.
2	LAMPORT LOGICAL CLOCK ALGORITHM.
3	BERKELEY CLOCK SYNCHRONIZATION ALGORITHM
4	MUTUAL EXCLUSION USING LAMPORT'S ALGORITHM.
5	RICART-AGRAWALA ALGORITHM FOR DISTRIBUTED MUTUAL EXCLUSION.
6	SIMPLE DISTRIBUTED CHAT APPLICATION USING SOCKETS.
7	DISTRIBUTED DEADLOCK DETECTION (WAIT-FOR GRAPH).
8	DISTRIBUTED SORTING (MERGE SORT / QUICK SORT) ACROSS NODES.
9	MAPREDUCE FOR WORD COUNT.
10	DISTRIBUTED PRIME NUMBER CALCULATION (RANGE SPLIT ACROSS NODES).

AIM: To demonstrate full-duplex, asynchronous message exchange between two independent network processes.

CODE:

messenger.py

```
import socket

import threading

def receive_loop(port):

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.bind(('0.0.0.0', port)); s.listen(5)

    while True:

        try:

            conn, _ = s.accept(); data = conn.recv(1024).decode().split(":", 1)

            print(f"\n[FROM {data[0]] {data[1]}\nMessage: ", end=""); conn.close()

        except: pass

def run_messenger():

    l_port = int(input("My Listen Port: ")); name = input("My Name: ")

    threading.Thread(target=receive_loop, args=(l_port,), daemon=True).start()

    r_ip, r_port = input("Peer IP: "), int(input("Peer Port: "))

    print("-" * 30); print(f"Full Duplex Mode Started. Type messages below.")

    while True:

        msg = input("Message: ")

        try:

            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

            s.settimeout(2); s.connect((r_ip, r_port)); s.sendall(f"{name}:{msg}".encode()); s.close()

        except: print("Peer is currently unreachable.")

if __name__ == "__main__": run_messenger()
```

OUTPUT:

<pre>PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/message_passing/messenger.py My Listen Port: 5001 My Name: B Peer IP: 10.58.2.253 Peer Port: 5000 ----- Full Duplex Mode Started. Type messages below. Message: [FROM A] hey Message: how ya doin'?</pre>	<pre>PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/message_passing/messenger.py My Listen Port: 5000 My Name: A Peer IP: 10.58.2.253 Peer Port: 5001 ----- Full Duplex Mode Started. Type messages below. Message: hey Message: [FROM B] how ya doin'?</pre>
--	--

RESULT:

Successfully implemented and executed the Full-Duplex Message Passing program.

AIM: To implement a logical clock system for consistent event ordering across different distributed nodes.

CODE:

process.py

```
import socket

import socket

import threading

class LamportNode:

    def __init__(self, port, name):

        self.port, self.name, self.clock, self.lock = port, name, 0, threading.Lock()

    def listen(self):

        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.bind(('0.0.0.0', self.port)); s.listen(5)

        while True:

            conn, _ = s.accept(); data = conn.recv(1024).decode().split(":")

            s_name, s_clock = data[0], int(data[1])

            with self.lock:

                self.clock = max(self.clock, s_clock) + 1

                print(f"\n[EVENT] '{s_name}' sent message. My New Clock: {self.clock}\nChoice (1-Event, 2-Send): ", end="")

            conn.close()

    def start(self):

        threading.Thread(target=self.listen, daemon=True).start()

        while True:

            choice = input("\n1: Internal Event, 2: Send Message\nChoice: ")

            if choice == '1':

                with self.lock: self.clock += 1; print(f"Internal Event. Clock: {self.clock}")

            elif choice == '2':

                t_ip, t_port = input("Target IP: "), int(input("Target Port: "))

                with self.lock: self.clock += 1; msg = f"{self.name}:{self.clock}"

                try:

                    sc = socket.socket(socket.AF_INET, socket.SOCK_STREAM); sc.connect((t_ip, t_port))

                    sc.sendall(msg.encode()); sc.close()

                except: print("Target unreachable.")

if __name__ == "__main__":
```

```
p = LamportNode(int(input("Listen Port: ")), input("Node Name: ")); p.start()
```

OUTPUT:

```
PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/lamport_mutex/node.py
My ID: 1
Node Name: A
Peer IDs: 2

Type 'req' for CS: [RECV] REQ from 'B'
[RECV] REL from 'B'
req
>>> IN CRITICAL SECTION <<<
>>> OUT OF CRITICAL SECTION <<<
Type 'req' for CS: █

PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/lamport_mutex/node.py
My ID: 2
Node Name: B
Peer IDs: 1

Type 'req' for CS: req
>>> IN CRITICAL SECTION <<<
>>> OUT OF CRITICAL SECTION <<<

Type 'req' for CS: [RECV] REQ from 'A'
[RECV] REL from 'A'
█
```

RESULT:

Successfully implemented and executed the Lamport Logical Clock algorithm.

AIM: To synchronize multiple physical clocks by calculating a master-slave average time and applying drift offsets.

CODE:**master.py**

```
import socket

import time

def run_master():

    port, count = int(input("Master Port: ")), int(input("Slave Count: "))

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.bind(('0.0.0.0', port)); s.listen(count)

    slaves = []; names = {}; print(f"Waiting for {count} slaves...")

    while len(slaves) < count:

        conn, addr = s.accept(); name = conn.recv(1024).decode()

        slaves.append(conn); names[conn] = name; print(f"[JOIN] Slave '{name}' connected from {addr}")

    times = [time.time()]

    for c in slaves:

        c.sendall(b"GET"); times.append(float(c.recv(1024).decode()))

    avg = sum(times) / len(times); print(f"Average System Time: {avg}")

    for i, c in enumerate(slaves):

        c.sendall(str(avg - times[i+1]).encode()); c.close()

    print("Sync finished."); s.close()

if __name__ == "__main__": run_master()
```

slave.py

```
import socket

import time

def run_slave():

    m_ip, m_port = input("Master IP: "), int(input("Master Port: "))

    name, drift = input("Slave Name: "), float(input("Drift (secs): "))

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.connect((m_ip, m_port))

    s.sendall(name.encode())

    while True:

        data = s.recv(1024).decode()

        if data == "GET": s.sendall(str(time.time() + drift).encode())

        else: print(f"Adjustment from Master: {data}"); break
```



```
s.close()

if __name__ == "__main__": run_slave()
```

OUTPUT:

```
PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/berkeley_sync/master.py
Master Port: 5000
Slave Count: 1
Waiting for 1 slaves...
[JOIN] Slave 'A' connected from ('10.58.2.253', 61087)
Average System Time: 1776453631.051772
Sync finished.
```

```
PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/berkeley_sync/slave.py
Master IP: 10.58.2.253
Master Port: 5000
Slave Name: A
Drift (secs): 4
Adjustment from Master: -2.000051975250244
PS D:\dheer@j\distributed_systems>
```

RESULT:

Successfully implemented and executed the Berkeley Clock Synchronization algorithm.

AIM: To ensure mutual exclusion using logical clocks and a shared request queue to prevent resource conflicts.

CODE:

node.py

```
import socket

import threading

import time

class Node:

    def __init__(self, id, name, peers):

        self.id, self.name, self.port, self.peers = id, name, 9000+id, peers

        self.clock, self.queue, self.replies = 0, [], 0

    def listen(self):

        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.bind(('0.0.0.0', self.port)); s.listen(5)

        while True:

            conn, _ = s.accept(); msg = conn.recv(1024).decode().split(':')

            t, s_id, s_name, s_clock = msg[0], int(msg[1]), msg[2], int(msg[3])

            self.clock = max(self.clock, s_clock) + 1

            if t == "REQ":

                print(f"[RECV] REQ from '{s_name}'"); self.queue.append((s_clock, s_id, s_name)); self.queue.sort()

                sc = socket.socket(socket.AF_INET, socket.SOCK_STREAM); sc.connect(('localhost', 9000+s_id))

                sc.sendall(f"REP:{self.id}:{self.name}:{self.clock}".encode()); sc.close()

            elif t == "REP": self.replies += 1

            elif t == "REL": print(f"[RECV] REL from '{s_name}'"); self.queue = [x for x in self.queue if x[1] != s_id]

            conn.close()

    def request_cs(self):

        self.clock += 1; self.replies = 0; self.queue.append((self.clock, self.id, self.name)); self.queue.sort()

        for p in self.peers:

            try:

                sc = socket.socket(socket.AF_INET, socket.SOCK_STREAM); sc.connect(('localhost', 9000+p))

                sc.sendall(f"REQ:{self.id}:{self.name}:{self.clock}".encode()); sc.close()

            except: pass

        while self.replies < len(self.peers) or self.queue[0][1] != self.id: time.sleep(0.1)

        print(">>> IN CRITICAL SECTION <<<"); time.sleep(2)
```

```

self.queue.pop(0); self.clock += 1

for p in self.peers:

    try:

        sc = socket.socket(socket.AF_INET, socket.SOCK_STREAM); sc.connect(('localhost', 9000+p))

        sc.sendall("#REL:{self.id}:{self.name}:{self.clock}".encode()); sc.close()

    except: pass

print(">>> OUT OF CRITICAL SECTION <<<")

if __name__ == "__main__":

    i, n = int(input("My ID: ")), input("Node Name: ")

    peers = [int(x) for x in input("Peer IDs: ").split()]

    node = Node(i, n, peers); threading.Thread(target=node.listen, daemon=True).start()

    while True:

        if input("\nType 'req' for CS: ") == 'req': node.request_cs()

```

OUTPUT:

<pre> PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/lamport_mutex/node.py My ID: 1 Node Name: A Peer IDs: 1 Type 'req' for CS: req [RECV] REQ from 'A' >>> IN CRITICAL SECTION <<< >>> OUT OF CRITICAL SECTION <<< [RECV] REL from 'A' Type 'req' for CS: [RECV] REQ from 'B' [RECV] REL from 'B' </pre>	<pre> PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/lamport_mutex/node.py My ID: 2 Node Name: B Peer IDs: 1 Type 'req' for CS: req >>> IN CRITICAL SECTION <<< >>> OUT OF CRITICAL SECTION <<< Type 'req' for CS: █ </pre>
--	--

RESULT:

Successfully implemented and executed the Mutual Exclusion using Lamport's algorithm.

AIM: To optimize distributed mutual exclusion by using a deferred-reply mechanism to reduce message overhead.

CODE:**node.py**

```
import socket

import threading

import time

class RANode:

    def __init__(self, id, name, peers):

        self.id, self.name, self.port, self.peers = id, name, 9000+id, peers

        self.clock, self.req_time, self.is_req, self.replies, self.deferred = 0, 0, False, 0, []

    def listen(self):

        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.bind(('0.0.0.0', self.port)); s.listen(5)

        while True:

            conn, _ = s.accept(); msg = conn.recv(1024).decode().split(':')

            t, s_id, s_name, s_clock = msg[0], int(msg[1]), msg[2], int(msg[3])

            self.clock = max(self.clock, s_clock) + 1

            if t == "REQ":

                prio = self.is_req and (self.req_time < s_clock or (self.req_time == s_clock and self.id < s_id))

                if prio: self.deferred.append((s_id, s_name))

            else:

                sc = socket.socket(socket.AF_INET, socket.SOCK_STREAM); sc.connect(('localhost', 9000+s_id))

                sc.sendall(f"REP:{self.id}:{self.name}:{self.clock}".encode()); sc.close()

            elif t == "REP": self.replies += 1

            conn.close()

    def request_cs(self):

        self.is_req, self.req_time, self.replies = True, self.clock + 1, 0

        for p in self.peers:

            try:

                sc = socket.socket(socket.AF_INET, socket.SOCK_STREAM); sc.connect(('localhost', 9000+p))

                sc.sendall(f"REQ:{self.id}:{self.name}:{self.req_time}".encode()); sc.close()

            except: pass
```

```

while self.replies < len(self.peers): time.sleep(0.1)

print(">>> IN CRITICAL SECTION <<<"); time.sleep(2)

print(">>> OUT OF CRITICAL SECTION <<<"); self.is_req = False

for s_id, _ in self.deferred:

    sc = socket.socket(socket.AF_INET, socket.SOCK_STREAM); sc.connect(('localhost', 9000+s_id))

    sc.sendall("#REP:{self.id}:{self.name}:{self.clock}".encode()); sc.close()

self.deferred = []

if __name__ == "__main__":

    i, n = int(input("My ID: ")), input("Node Name: ")

    peers = [int(x) for x in input("Peer IDs: ").split()]

    node = RANode(i, n, peers); threading.Thread(target=node.listen, daemon=True).start()

    while True:

        if input("\nType 'req' for CS: ") == 'req': node.request_cs()

```

OUTPUT:

<pre> PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/ricart_agrawala/node.py My ID: 1 Node Name: A Peer IDs: 2 Type 'req' for CS: req >>> IN CRITICAL SECTION <<< >>> OUT OF CRITICAL SECTION <<< Type 'req' for CS: req >>> IN CRITICAL SECTION <<< >>> OUT OF CRITICAL SECTION <<< </pre>	<pre> PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/ricart_agrawala/node.py My ID: 2 Node Name: B Peer IDs: 1 Type 'req' for CS: req >>> IN CRITICAL SECTION <<< >>> OUT OF CRITICAL SECTION <<< Type 'req' for CS: </pre>
--	---

RESULT

Successfully implemented and executed the Ricart–Agrawala algorithm for distributed mutual exclusion.

SIMPLE DISTRIBUTED CHAT APPLICATION USING SOCKETS.

AIM: To build a multi-client system that broadcasts messages through a central server with real-time user logging.

CODE:**server.py**

```
import socket

import threading

clients = {}

def handle(conn, addr):

    try:

        name = conn.recv(1024).decode(); clients[conn] = name

        print(f"[JOIN] User '{name}' ({addr}) connected.")

        while True:

            data = conn.recv(1024)

            if not data: break

            msg = data.decode(); print(f"[{name}] {msg}")

            for c, n in clients.items():

                if c != conn: c.sendall(f"{n}: {msg}".encode())

        except: pass

        name = clients.pop(conn, "Unknown"); print(f"[LEAVE] User '{name}' disconnected."); conn.close()

def run_server():

    port = int(input("Listen Port: ")); s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    s.bind(('0.0.0.0', port)); s.listen(5); print(f"Chat Server started on {port}...")

    while True:

        conn, addr = s.accept()

        threading.Thread(target=handle, args=(conn, addr), daemon=True).start()

if __name__ == "__main__": run_server()
```

client.py

```
import socket

import threading

def receive_msgs(s):

    while True:

        try:
```

```

        msg = s.recv(1024).decode()
        if not msg: break
        print(f"\n{msg}\nInput: ", end="")

    except: break

def run_client():

    ip = input("Server IP: "); port = int(input("Server Port: "))
    name = input("Enter your Username: ")
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:

        s.connect((ip, port)); s.sendall(name.encode())
        print(f"Connected to Chat as '{name}'")
        threading.Thread(target=receive_msgs, args=(s,), daemon=True).start()

        while True:

            msg = input("Input: "); s.sendall(msg.encode())

            if msg.lower() == 'exit': break

    except Exception as e: print(f"Error: {e}")

    finally: s.close()

if __name__ == "__main__": run_client()

```

OUTPUT:

<pre> PS D:\dheer@j\distributed_system s> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/chat_app/server.py Listen Port: 5000 Chat Server started on 5000... [JOIN] User 'a' (('10.58.2.253', 52364)) connected. [JOIN] User 'b' (('10.58.2.253', 52372)) connected. [b] hey [a] hi </pre>	<pre> PS D:\dheer@j\distributed_system s> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/chat_app/client.py Server IP: 10.58.2.253 Server Port: 5000 Enter your Username: a Connected to Chat as 'a' Input: b: hey Input: hi Input: </pre>	<pre> PS D:\dheer@j\distributed_system s> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/chat_app/client.py Server IP: 10.58.2.253 Server Port: 5000 Enter your Username: b Connected to Chat as 'b' Input: hey Input: a: hi Input: </pre>
--	---	---

RESULT

Successfully implemented and executed the simple Distributed Chat Application using sockets.

AIM: To identify circular wait conditions in a system by maintaining and analyzing a distributed wait-for-graph.

CODE:**director.py**

```
import socket

def run_detector():

    port = int(input("Listen Port: ")); s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    s.bind(('0.0.0.0', port)); s.listen(5); graph = {}; print(f"Deadlock Detector started on {port}...")

    def has_cycle(u, visited, path):

        visited.add(u); path.add(u)

        for v in graph.get(u, []):

            if v not in visited:

                if has_cycle(v, visited, path): return True

            elif v in path: return True

        path.remove(u); return False

    while True:

        conn, _ = s.accept(); u, v = conn.recv(1024).decode().split(",")

        graph.setdefault(u, []).append(v); print(f"[UPDATE] Edge Added: {u} -> {v}")

        visted, path, deadlock = set(), set(), False

        for node in list(graph.keys()):

            if node not in visted and has_cycle(node, visted, path): deadlock = True; break

        if deadlock: print(">>> ALERT: DEADLOCK DETECTED IN THE SYSTEM <<<")

        conn.close()

if __name__ == "__main__": run_detector()
```

node.py

```
import socket

def run_node():

    d_ip, d_port = input("Detector IP: "), int(input("Detector Port: "))

    name = input("My Node Name: ")

    while True:

        target = input("I am waiting for (Node Name): ")

        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.connect((d_ip, d_port))
```



```

s.sendall(f'{name},{target}'.encode()); s.close()

if input("Add another dependency? (y/n): ") == 'n': break

if __name__ == "__main__": run_node()

```

OUTPUT:

<pre> PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/deadlock_detection/detector.py Listen Port: 5000 Deadlock Detector started on 5000... [UPDATE] Edge Added: A -> B [UPDATE] Edge Added: A -> C [UPDATE] Edge Added: A -> A >>> ALERT: DEADLOCK DETECTED IN THE SYSTEM <<< █ </pre>	<pre> PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/deadlock_detection/node.py Detector IP: 10.58.2.253 Detector Port: 5000 My Node Name: A I am waiting for (Node Name): B Add another dependency? (y/n): y I am waiting for (Node Name): C Add another dependency? (y/n): y I am waiting for (Node Name): A Add another dependency? (y/n): n ○ PS D:\dheer@j\distributed_systems> █ </pre>
--	---

RESULT

Successfully implemented and executed the Distributed Deadlock Detection (wait-for graph) program.

AIM: To perform parallel sorting by splitting data chunks across multiple nodes and merging the sorted parts.

CODE:**master.py**

```
import socket
import pickle

def run_master():
    port, count = int(input("Master Port: "), int(input("Worker Count: "))
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.bind(('0.0.0.0', port)); s.listen(count)
    nums = [int(x) for x in input("Numbers to sort: ").split()]; step = len(nums)//count; results = []
    print(f"Waiting for {count} workers...")
    for i in range(count):
        conn, _ = s.accept(); name = conn.recv(1024).decode()
        chunk = nums[i*step:(i+1)*step] if i<count-1 else nums[i*step:]
        print(f"[TASK] Sending chunk to Worker '{name}'"); conn.sendall(pickle.dumps(chunk))
        results.extend(pickle.loads(conn.recv(4096))); conn.close()
    print(f"Final Sorted Result: {sorted(results)}"); s.close()

if __name__ == "__main__": run_master()
```

worker.py

```
import socket
import pickle

def run_worker():
    m_ip, m_port = input("Master IP: "), int(input("Master Port: "))
    name = input("Worker Name: ")
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.connect((m_ip, m_port))
    s.sendall(name.encode()); data = pickle.loads(s.recv(4096))
    print(f"Sorting chunk of size {len(data)}..."); data.sort(); s.sendall(pickle.dumps(data)); s.close()

if __name__ == "__main__": run_worker()
```

OUTPUT:

```
PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/distributed_sorting/master.py
Master Port: 6000
Worker Count: 1
Numbers to sort: 7 4 3 6 8 45 1 23 1 0 9 5 3 4
Waiting for 1 workers...
[TASK] Sending chunk to Worker 'A'
Final Sorted Result: [0, 1, 1, 3, 3, 4, 4, 5, 6, 7, 8, 9, 23, 45]
```

```
PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/distributed_sorting/worker.py
Master IP: 10.58.2.253
Master Port: 6000
Worker Name: A
Sorting chunk of size 14...
PS D:\dheer@j\distributed_systems>
```

RESULT

Successfully implemented and executed the Distributed Sorting across nodes.

AIM: To simulate large-scale processing by parallelizing "map" (local counting) and "reduce" (global aggregation) operations.

CODE:**master.py**

```
import socket

import pickle

def run_master():

    port, count = int(input("Master Port: "), int(input("Worker Count: "))

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.bind(('0.0.0.0', port)); s.listen(count)

    text = input("Sentence: ").split(); step = len(text)//count; final_counts = {}

    print(f"Waiting for {count} workers...")

    for i in range(count):

        conn, _ = s.accept(); name = conn.recv(1024).decode()

        chunk = text[i*step:(i+1)*step] if i<count-1 else text[i*step:]

        print(f"[MAP] Sending words to Worker '{name}'"); conn.sendall(pickle.dumps(chunk))

        res = pickle.loads(conn.recv(4096))

        for k, v in res.items(): final_counts[k] = final_counts.get(k, 0) + v

        conn.close()

    print(f"Final Word Counts: {final_counts}"); s.close()

if __name__ == "__main__": run_master()
```

worker.py

```
import socket

import pickle

def run_worker():

    m_ip, m_port = input("Master IP: "), int(input("Master Port: "))

    name = input("Worker Name: ")

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.connect((m_ip, m_port))

    s.sendall(name.encode()); words = pickle.loads(s.recv(4096)); res = {}

    for w in words: res[w] = res.get(w, 0) + 1

    print(f"Mapped {len(words)} words."); s.sendall(pickle.dumps(res)); s.close()

if __name__ == "__main__": run_worker()
```

OUTPUT:

```
PS D:\dheer@j\distributed_systems> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/mapreduce/master.py
Master Port: 5000
Worker Count: 2
Sentence: I WILL BE SHOWING YOU SOMETHING BY I WON
T BR SHOWING YOU THAT YET
Waiting for 2 workers...
[MAP] Sending words to Worker 'A'
[MAP] Sending words to Worker 'B'
Final Word Counts: {'I': 2, 'WILL': 1, 'BE': 1, 'S
HOWING': 2, 'YOU': 2, 'SOMETHING': 1, 'BY': 1, 'WO
NT': 1, 'BR': 1, 'THAT': 1, 'YET': 1}
PS D:\dheer@j\distributed_systems>

Worker Name: A PS D:\dheer@j\distributed_systems>
& C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/mapreduce/WORKER.PY
Master IP: 10.58.2.253
Master Port: 5000
Worker Name: A
Mapped 7 words.
PS & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/mapreduce/WORKER.PY
Master IP: 10.58.2.253
Master Port: 5000
Worker Name: B
Mapped 7 words.
PS D:\dheer@j\distributed_systems>
```

RESULT

Successfully implemented and executed the MapReduce simulation for Word Count.

Practical No: 10**DISTRIBUTED PRIME NUMBER CALCULATION
(RANGE SPLIT ACROSS NODES).**

AIM: To calculate prime numbers efficiently by splitting a massive search range across multiple computational workers.

CODE:**master.py**

```
import socket
import pickle

def run_master():
    port, count = int(input("Master Port: ")), int(input("Worker Count: "))
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.bind(('0.0.0.0', port)); s.listen(count)
    start, end = int(input("Start: ")), int(input("End: "))
    step, primes = (end-start)//count, []
    for i in range(count):
        conn, _ = s.accept(); name = conn.recv(1024).decode()
        p_start = start + i*step; p_end = (start + (i+1)*step) if i<count-1 else end
        print(f"[TASK] Assigning range {p_start}-{p_end} to Worker '{name}'")
        conn.sendall(pickle.dumps((p_start, p_end))); primes.extend(pickle.loads(conn.recv(4096))); conn.close()
    print(f"All Primes: {primes}"); s.close()

if __name__ == "__main__": run_master()
```

worker.py

```
import socket
import pickle

def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0: return False
    return True

def run_worker():
    m_ip, m_port = input("Master IP: "), int(input("Master Port: "))
    name = input("Worker Name: ")
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.connect((m_ip, m_port)); s.sendall(name.encode())
    r_start, r_end = pickle.loads(s.recv(4096)); res = [x for x in range(r_start, r_end) if is_prime(x)]
    print(f"Found {len(res)} primes."); s.sendall(pickle.dumps(res)); s.close()

if __name__ == "__main__": run_worker()
```

OUTPUT:

<pre>PS D:\dheer@j\distributed_system s> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/prime_calculation/master.py Master Port: 9000 Worker Count: 2 Start: 1 End: 1000 [TASK] Assigning range 1-500 to Worker 'A' [TASK] Assigning range 500-1000 to Worker 'B' All Primes: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109,</pre>	<pre>PS D:\dheer@j\distributed_system s> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/prime_calculation/worker.py Master IP: 10.58.2.253 Master Port: 9000 Worker Name: A Found 95 primes. PS D:\dheer@j\distributed_system s></pre>	<pre>PS D:\dheer@j\distributed_system s> & C:/Users/dheer/AppData/Local/Programs/Python/Python313/python.exe d:/dheer@j/distributed_systems/DS_LAB/prime_calculation/worker.py Master IP: 10.58.2.253 Master Port: 9000 Worker Name: B Found 73 primes. PS D:\dheer@j\distributed_system s></pre>
--	---	---

RESULT:

Successfully implemented and executed the Distributed Prime Number Calculation (range splitting).